

digital Tabernacle: Grace Book Master File (V2)

1. Project Overview & Scope

Grace book is a specialized social networking platform built for community building, fellowship, and discipleship. This document serves as the absolute master file containing all codebase components, architecture configurations, schemas, and structural deployment roadmaps generated from the project's inception.

- **Domain Name:** gracebookdisciples.com
- **Target Environment:** Local development transitioning to a live production server hosted via Render and continuously deployed via GitHub.
- **Core Tech Stack:** Node.js, Express framework, Sequelize ORM, and a PostgreSQL relational database.

2. Database Schema & Architecture

The backend architecture uses a relational database model structured to efficiently isolate user accounts, enforce security records, and handle social interactions seamlessly.

Conceptual SQL Schema

```
-- Master Relational Database Schema
CREATE TABLE Users (
  id SERIAL PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Posts (
  id SERIAL PRIMARY KEY,
  user_id INT REFERENCES Users(id) ON DELETE CASCADE,
  content TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

3. Complete Codebase Implementation

The following scripts contain the complete logic for database object-relational mapping models,

environment setup foundations, and authentication management routing.

Sequelize Object-Relational Models

User Model (User.js)

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const User = sequelize.define('User', {
  username: {
    type: DataTypes.STRING,
    allowNull: false,
    unique: true
  },
  email: {
    type: DataTypes.STRING,
    allowNull: false,
    unique: true,
    validate: { isEmail: true }
  },
  password: {
    type: DataTypes.STRING,
    allowNull: false
  }
}, {
  timestamps: true
});

module.exports = User;
```

Post Model (Post.js)

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const Post = sequelize.define('Post', {
  content: {
    type: DataTypes.TEXT,
    allowNull: false
  },
  userId: {
    type: DataTypes.INTEGER,
    allowNull: false,
    references: {
      model: 'Users',
      key: 'id'
    }
  }
});
```

```

    }
  }, {
    timestamps: true
  });

module.exports = Post;

```

Authentication Controller (authController.js)

Manages user profile initialization, registration requests, password encryption protocols, and tokenized session authorization handlers.

```

const User = require('../models/User');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

exports.register = async (req, res) => {
  try {
    const { username, email, password } = req.body;
    const hashedPassword = await bcrypt.hash(password, 10);
    const newUser = await User.create({ username, email, password:
hashedPassword });
    res.status(201).json({ message: "User registered
successfully", userId: newUser.id });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ where: { email } });
    if (!user) return res.status(404).json({ error: "User not
found" });

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) return res.status(401).json({ error: "Invalid
credentials" });

    const token = jwt.sign({ id: user.id },
process.env.JWT_SECRET, { expiresIn: '1d' });
    res.status(200).json({ message: "Login successful", token });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

```

4. Render & Deployment Environment Configuration

To successfully transition from a local development environment into a live state reality, the application interfaces with Render using GitHub for version tracking. The steps below detail your production environment requirements:

- **GitHub Linkage:** Project source code must be housed in a private or public GitHub repository connected to your Render account profile.
- **Render Web Service Setup:** Web service environment should target the Node runtime environment with npm start as the active execution trigger.
- **Environment Variables:** The live Render instances require an explicitly defined key-value store containing DATABASE_URL (pointing to the provisioned PostgreSQL instance) and JWT_SECRET (used for authentication session signatures).

5. Professional Portfolio Documentation (README.md Template)

The core framework for the professional portfolio project summary file, compiled to exhibit proper architectural planning to external reviewers.

Grace Book

A specialized web application platform engineered for social connection, community fellowship, and structured digital discipleship.

Core Features

- Full secure JWT token-based authentication workflow.
- Relational database storage isolating users, data structures, and post creation history.
- Scalable configuration setups built ready for simple production server pipelines.

Technology Stack

- **Runtime:** Node.js
- **Framework:** Express.js
- **ORM:** Sequelize (v6)
- **Database:** PostgreSQL

Installation & Setup

1. Clone the repository locally.
2. Execute `npm install` to load core dependencies.
3. Configure your local `.env` setup file.
4. Run `npm run dev` to launch the active development listener.

6. Month 1 Structural Development Roadmap

A strategic breakdown tracking the exact timeline targets from initial file structure initialization to live server monitoring verification.

Phase	Focus Area	Key Objectives
Week 1	Environment Setup	Initialize local Git repository, set up local PostgreSQL database, configure Express server structure.
Week 2	Core Backend Development	Implement User and Post Sequelize schemas, establish password hashing and JWT authentication routes.
Week 3	Deployment Pipeline	Create a GitHub repository, push code online, link GitHub to Render hosting, provision online PostgreSQL instance.
Week 4	API Testing & Portfolio	Verify API endpoints using Postman/Insomnia, finalize the professional portfolio README.md file.